CROSS COMPILERS FOR EMBEDDED SYSTEMS

Cross compilers are devices that are capable of manufacturing executable code that can be run on a platform that is currently not the resident platform for the compiler. The utilization of a cross compiler is common when there is a need to make use of multiple platforms in order to handle computing functions. This will include embedded systems where each embedded computer within the system has a smaller amount of resources. The use of a cross compiler makes it possible to overcome this lack of resources by creating an interrelated execution between various components on the system.

In embedded systems compilers are referred based on processor architecture and not on OS. What you probably want is to build a "cross compiler" for a processor other than the one running in your Windows machine.

Uses of cross compilers

The fundamental use of a cross compiler is to separate the build environment from target environment. This is useful in a number of situations:

- Embedded computers where a device has extremely limited resources. For example, a microwave oven will have an extremely small computer to read its touchpad and door sensor, provide output to a digital display and speaker, and to control the machinery for cooking food. This computer will not be powerful enough to run a compiler, a file

system, or a development environment. Since debugging and testing may also require more resources than are available on an embedded system, cross-compilation can be less involved and less prone to errors than native compilation.

- Compiling for multiple machines. For example, a company may wish to support several different versions of an operating system or to support several different operating systems. By using a cross compiler, a single build environment can be set up to compile for each of these targets.
- Compiling on a server farm. Similar to compiling for multiple machines, a complicated build that involves many compile operations can be executed across any machine that is free, regardless of its underlying hardware or the operating system version that it is running.
- Bootstrapping to a new platform. When developing software for a new platform, or the emulator of a future platform, one uses a cross compiler to compile necessary tools such as the operating system and a native compiler.
- Compiling native code for emulators for older now-obsolete platforms like the Commodore 64 or Apple II by enthusiasts who use cross compilers that run on a current platform (such as Aztec C's MS DOS 6502 cross compilers running under Windows XP).

Use of virtual machines (such as Java's JVM) resolves some of the reasons for which cross compilers were developed. The virtual machine paradigm allows the same compiler output to be used across multiple target systems.

Typically the hardware architecture differs (e.g. compiling a program destined for the MIPS

Typically the hardware architecture differs (e.g. compiling a program destined for the MIPS architecture on an x86 computer) but cross-compilation is also applicable when only the operating system environment differs, as when compiling a FreeBSD program under Linux, or even just the system library, as when compiling programs with uClibc on a glibc host.